Object-Oriented and Classical Software Engineering

8th Edition, WCB/McGraw-Hill, 2010

Stephen R. Schach
srs@vuse.vanderbilt.edu

CHAPTER 6

TESTING

Overview

- Quality issues
- Non-execution-based testing
- Execution-based testing
- What should be tested?
- Testing versus correctness proofs
- Who should perform execution-based testing?
- When testing stops

Testing

- There are two basic types of testing
  - Execution-based testing
  - Non-execution-based testing

Testing (contd)

- "V & V"
  - *Verification*
    - Determine if the workflow was completed correctly
  - *Validation*
    - Determine if the product as a whole satisfies its requirements

Testing (contd)

- Warning
  - The term “verify” is also used for all non-execution-based testing

Software Quality

- Not “excellence”
- The extent to which software satisfies its specifications
- Every software professional is responsible for ensuring that his or her work is correct
  - Quality must be built in from the beginning

Software Quality Assurance

- The members of the SQA group must ensure that the developers are doing high-quality work
  - At the end of each workflow
  - When the product is complete
- In addition, quality assurance must be applied to
  - The process itself
    - Example: Standards

Managerial Independence

- There must be managerial independence between
  - The development group
  - The SQA group
- Neither group should have power over the other

Managerial Independence (contd)

- More senior management must decide whether to
  - Deliver the product on time but with faults, or
  - Test further and deliver the product late
- The decision must take into account the interests of the client and the development organization

Non-Execution-Based Testing

- Underlying principles
  - We should not review our own work
  - Group synergy

Walkthroughs

- A walkthrough team consists of from four to six members
- It includes representatives of
  - The team responsible for the current workflow
  - The team responsible for the next workflow
  - The SQA group
- The walkthrough is preceded by preparation
  - Lists of items
    - Items not understood
    - Items that appear to be incorrect

Managing Walkthroughs

- The walkthrough team is chaired by the SQA representative
- In a walkthrough we detect faults, not correct them
  - A correction produced by a committee is likely to be of low quality
  - The cost of a committee correction is too high
  - Not all items flagged are actually incorrect
  - A walkthrough should not last longer than 2 hours
  - There is no time to correct faults as well

Managing Walkthroughs (contd)

- A walkthrough must be document-driven, rather than participant-driven
- Verbalization leads to fault finding
- A walkthrough should never be used for performance appraisal

Inspections

- An inspection has five formal steps
  - Overview
  - Preparation, aided by statistics of fault types
  - Inspection
  - Rework
  - Follow-up

Inspections (contd)

- An inspection team has four members
  - Moderator
  - A member of the team performing the current workflow
  - A member of the team performing the next workflow
  - A member of the SQA group

- Special roles are played by the
  - Moderator
  - Reader
  - Recorder

Fault Statistics

- Faults are recorded by severity
  - Example:
    - Major or minor
- Faults are recorded by fault type
  - Examples of design faults:
    - Not all specification items have been addressed
    - Actual and formal arguments do not correspond

Fault Statistics (contd)

- For a given workflow, we compare current fault rates with those of previous products
- We take action if there are a disproportionate number of faults in an artifact
  - Redesigning from scratch is a good alternative
- We carry forward fault statistics to the next workflow
  - We may not detect all faults of a particular type in the current inspection

Statistics on Inspections

- IBM inspections showed up
  - 82% of all detected faults (1976)
  - 70% of all detected faults (1978)
  - 93% of all detected faults (1986)
- Switching system
  - 90% decrease in the cost of detecting faults (1986)
- JPL
  - Four major faults, 14 minor faults per 2 hours (1990)
  - Savings of $25,000 *per inspection*
  - The number of faults decreased exponentially by phase (1992)

Statistics on Inspections (contd)

- Warning
- Fault statistics should never be used for performance appraisal
  - “Killing the goose that lays the golden eggs”

Comparison of Inspections and Walkthroughs

- Walkthrough
  - Two-step, informal process
    - Preparation
    - Analysis
- Inspection
  - Five-step, formal process
    - Overview
    - Preparation
    - Inspection
    - Rework
    - Follow-up

Strengths and Weaknesses of Reviews

- Reviews can be effective
  - Faults are detected early in the process
- Reviews are less effective if the process is inadequate
  - Large-scale software should consist of smaller, largely independent pieces
  - The documentation of the previous workflows has to be complete and available online

Metrics for Inspections

- Inspection rate (e.g., design pages inspected per hour)
- Fault density (e.g., faults per KLOC inspected)
- Fault detection rate (e.g., faults detected per hour)
- Fault detection efficiency (e.g., number of major, minor faults detected per hour)

Metrics for Inspections (contd)

- Does a 50% increase in the fault detection rate mean that
  - Quality has decreased? Or
  - The inspection process is more efficient?

Execution-Based Testing

- Organizations spend up to 50% of their software budget on testing
  - But delivered software is frequently unreliable
- Dijkstra (1972)
  - “Program testing can be a very effective way to show the presence of bugs, but it is hopelessly inadequate for showing their absence”

What Should Be Tested?

- Definition of *execution-based testing*
  - “The process of inferring certain behavioral properties of the product based, in part, on the results of executing the product in a known environment with selected inputs”
- This definition has troubling implications

What Should Be Tested? (contd)

- “Inference”
  - We have a fault report, the source code, and — often — nothing else
- “Known environment”
  - We never can really know our environment
- “Selected inputs”
  - Sometimes we cannot provide the inputs we want
  - Simulation is needed

What Should Be Tested? (contd)

- We need to test correctness (of course), and also
  - Utility
  - Reliability
  - Robustness, and
  - Performance

Utility

- The extent to which the product meets the user's needs
  - Examples:
    - Ease of use
    - Useful functions
    - Cost effectiveness

Reliability

- A measure of the frequency and criticality of failure
  - Mean time between failures
  - Mean time to repair
  - Time (and cost) to repair the *results* of a failure

Robustness

- A function of
  - The range of operating conditions
  - The possibility of unacceptable results with valid input
  - The effect of invalid input

Performance

- The extent to which space and time constraints are met
- Real-time software is characterized by *hard* real-time constraints
- If data are lost because the system is too slow
  - There is no way to recover those data

Correctness

- A product is correct if it satisfies its specifications

Correctness of specifications

Input specification:	p : array of n integers, $n > 0$.
Output specification:	q : array of n integers such that $q[0] \leq q[1] \leq \cdots \leq q[n-1]$

-
```
void trickSort (int p[ ], int q[ ])
{
    int i;
    for (i = 0; i < n; i++)
        q[i] = 0;
}
```

Correctness of specifications (contd)

- Incorrect specification for a sort:

Input specification:	p : array of n integers, $n > 0$.
Output specification:	q : array of n integers such that $q[0] \leq q[1] \leq \cdots \leq q[n-1]$

- Corrected specification for the sort:

Input specification:	p : array of n integers, $n > 0$.
Output specification:	q : array of n integers such that $q[0] \leq q[1] \leq \cdots \leq q[n-1]$ The elements of array q are a permutation of the elements of array p, which are unchanged.

Correctness (contd)

- Technically, correctness is
- *Not* necessary
  - Example: C++ compiler
- *Not* sufficient
  - Example: `trickSort`

Testing versus Correctness Proofs

- A correctness proof is an alternative to execution-based testing

Correctness Proofs and Software Engineering

- Three myths of correctness proving (see over)

Three Myths of Correctness Proving

- Software engineers do not have enough mathematics for proofs
  - Most computer science majors either know or can learn the mathematics needed for proofs
- Proving is too expensive to be practical
  - Economic viability is determined from cost–benefit analysis
- Proving is too hard
  - Many nontrivial products have been successfully proven
  - Tools like theorem provers can assist us

Difficulties with Correctness Proving

- Can we trust a theorem prover ?

```
void theoremProver ( )
{
    print "This product is correct";
}
```

Difficulties with Correctness Proving (contd)

- How do we find input–output specifications, loop invariants?
- What if the specifications are wrong?
- We can never be sure that specifications or a verification system are correct

Correctness Proofs and Software Engineering (contd)

- Correctness proofs are a vital software engineering tool, *where appropriate*:
  - When human lives are at stake
  - When indicated by cost–benefit analysis
  - When the risk of not proving is too great

- Also, informal proofs can improve the quality of the product
  - Use the `assert` statement

Who Should Perform Execution-Based Testing?

- Programming is *con*structive
- Testing is *de*structive
  - A successful test finds a fault
- So, programmers should not test their own code artifacts

Who Should Perform Execution-Based Testing? (contd)

- Solution:
  - The programmer does informal testing
  - The SQA group then does systematic testing
  - The programmer debugs the module

- All test cases must be
  - Planned beforehand, including the expected output, and
  - Retained afterwards

When Testing Stops

- Only when the product has been irrevocably discarded